SafeDocs

Secure PDF Management System

CS 432 – Databases
Course Project: Assignment 1
Track 1 | Statement 6

Semester II (2025–2026)

Indian Institute of Technology, Gandhinagar

Name	Roll Number
Sujal Patel	22110261
Siddharth Doshi	22110250
Viraj Vekaria	22110287
Nishit Mistry	22110172
Rutvi Shah	22110227

GitHub Repository: https://github.com/SujalAVMP/SafeDocs

Instructor: Dr. Yogesh K. Meena

Date: February 15, 2026

Contents

1 Introduction **2**
- 1.1 Problem Statement . . . 2
- 1.2 Motivation . . . 2
- 1.3 Scope and Core Functionalities . . . 2

2 Module A: Database Design and Implementation **2**
- 2.1 Entity Overview . . . 2
- 2.2 Relational Schema (All 12 Tables) . . . 3
  - 2.2.1 Department . . . 3
  - 2.2.2 Role . . . 4
  - 2.2.3 Member . . . 4
  - 2.2.4 Folder . . . 5
  - 2.2.5 Document . . . 6
  - 2.2.6 DocumentVersion . . . 7
  - 2.2.7 Tag . . . 7
  - 2.2.8 DocumentTag . . . 7
  - 2.2.9 Permission . . . 8
  - 2.2.10 AccessLog . . . 9
  - 2.2.11 SharedLink . . . 10
  - 2.2.12 Notification . . . 11
- 2.3 Summary of Constraints . . . 11
- 2.4 Key Logical Constraints . . . 11
- 2.5 Normalisation . . . 12

3 Module B: Conceptual Design (UML and ER Diagrams) **12**
- 3.1 UML Class Diagram . . . 12
  - 3.1.1 UML Relationship Summary . . . 14
- 3.2 ER Diagram . . . 14
- 3.3 UML-to-ER Transition Explanation . . . 15
- 3.4 Relationship Justifications . . . 16

4 Additional Constraints and Design Decisions **16**

5 Sample Data Overview **17**

6 Team Member Contributions **18**

7 References **18**

1 Introduction

1.1 Problem Statement

Organisations handle large volumes of sensitive PDF documents—legal files, financial reports, academic records, and internal policies. These are often stored in shared folders, email attachments, or unsecured cloud drives, making them vulnerable to unauthorised access, accidental deletion, data leaks, and compliance violations. There is little visibility into who accessed which file, when, or what actions were performed.

1.2 Motivation

Existing solutions like Google Drive and Dropbox provide basic sharing but lack fine-grained, per-document permission controls (view-only, edit, delete restrictions), comprehensive audit trails, and mechanisms to reject or block unwanted files. **SafeDocs** addresses these gaps by providing:

- Fine-grained, per-document access control
- Immutable audit logging of every interaction
- Secure, expirable shared links with download limits
- Version tracking for document history
- Folder-based and tag-based organisation with department scoping
- User notifications for document events

1.3 Scope and Core Functionalities

The system supports **six core functionalities**:

1. **Document Upload & Version Control** – Upload PDFs, maintain a full version history with change notes.
2. **Fine-Grained Access Control** – Per-document permissions (VIEW, EDIT, DELETE, ADMIN) with optional expiry dates.
3. **Audit Logging & Compliance** – Immutable logs tracking every view, download, edit, share, and permission change.
4. **Secure Document Sharing** – Generate tokenised links with configurable expiry and maximum download counts.
5. **Document Organisation & Search** – Hierarchical folders, tags, and department-scoped organisation.
6. **Notification System** – Alerts for shares, permission changes, version updates, and system events.

2 Module A: Database Design and Implementation

2.1 Entity Overview

The system comprises **7 primary entities** realised across **12 tables** (including junction and log tables):

Table 1: Entity and Table Mapping

#	Table	Entity Type	Purpose
1	Department	Strong Entity	Organisational units
2	Role	Strong Entity	User roles with capability flags
3	Member	Strong Entity	System users (mandatory table)
4	Folder	Strong Entity	Hierarchical document folders
5	Document	Strong Entity	Core PDF document metadata
6	DocumentVersion	Weak Entity	Version history per document
7	Tag	Strong Entity	Categorisation labels
8	DocumentTag	Junction Table	M:N link between Document and Tag
9	Permission	Relationship Entity	Per-document access grants
10	AccessLog	Log Entity	Immutable audit trail
11	SharedLink	Strong Entity	Secure external sharing links
12	Notification	Strong Entity	User alert messages

2.2 Relational Schema (All 12 Tables)

Below is the complete schema for each table. Underlined attributes denote Primary Keys. Attributes marked with (FK) are Foreign Keys. Constraints are listed under each table.

2.2.1 Department

Attribute	Type	Constraints	Description
DepartmentID	INT	PK, AUTO_INCREMENT	Unique department identifier
DeptName	VARCHAR(100)	NOT NULL, UNIQUE	Department name
Description	VARCHAR(255)	–	Optional description
CreatedAt	DATETIME	NOT NULL, DEFAULT NOW	Timestamp of creation

2.2.2 Role

Attribute	Type	Constraints	Description
RoleID	INT	PK, AUTO_INCREMENT	Unique role identifier
RoleName	VARCHAR(50)	NOT NULL, UNIQUE	Role label
Description	VARCHAR(255)	–	Optional description
CanUpload	BOOLEAN	NOT NULL, DEFAULT FALSE	Upload capability
CanDelete	BOOLEAN	NOT NULL, DEFAULT FALSE	Delete capability
CanShare	BOOLEAN	NOT NULL, DEFAULT FALSE	Share capability
CanManageUsers	BOOLEAN	NOT NULL, DEFAULT FALSE	User management capability

2.2.3 Member

Attribute	Type	Constraints	Description
MemberID	INT	PK, AUTO_INCREMENT	Unique member identifier
Name	VARCHAR(100)	NOT NULL	Full name
Image	VARCHAR(255)	–	Profile image path
Age	INT	CHECK ≥ 18	Age of the member
Email	VARCHAR(150)	NOT NULL, UNIQUE	Email address
ContactNumber	VARCHAR(15)	NOT NULL, UNIQUE	Phone number
DepartmentID	INT	NOT NULL, FK $\rightarrow$ Department	Department reference
RoleID	INT	NOT NULL, FK $\rightarrow$ Role	Role reference
PasswordHash	VARCHAR(255)	NOT NULL	Hashed password
IsActive	BOOLEAN	NOT NULL, DEFAULT TRUE	Account status
CreatedAt	DATETIME	NOT NULL, DEFAULT NOW	Registration timestamp

Referential Integrity:

- `DepartmentID`: ON UPDATE CASCADE, ON DELETE RESTRICT

- `RoleID`: ON UPDATE CASCADE, ON DELETE RESTRICT

2.2.4 Folder

Attribute	Type	Constraints	Description
FolderID	INT	PK, AUTO_INCREMENT	Unique folder identifier
FolderName	VARCHAR(100)	NOT NULL	Folder display name
Description	VARCHAR(255)	–	Optional description
ParentFolderID	INT	FK → Folder (self)	Parent for nesting
CreatedBy	INT	NOT NULL, FK → Member	Creator reference
DepartmentID	INT	FK → Department	Optional dept. scoping
CreatedAt	DATETIME	NOT NULL, DEFAULT NOW	Creation timestamp
IsActive	BOOLEAN	NOT NULL, DEFAULT TRUE	Soft-delete flag

Referential Integrity:

- `ParentFolderID`: ON UPDATE CASCADE, ON DELETE SET NULL (orphaned sub-folders become root)
- `CreatedBy`: ON UPDATE CASCADE, ON DELETE RESTRICT
- `DepartmentID`: ON UPDATE CASCADE, ON DELETE SET NULL

2.2.5 Document

Attribute	Type	Constraints	Description
DocumentID	INT	PK, AUTO_INCREMENT	Unique document identifier
Title	VARCHAR(200)	NOT NULL	Document title
Description	VARCHAR(500)	–	Optional summary
FilePath	VARCHAR(500)	NOT NULL	Storage path
FileSize	BIGINT	NOT NULL, CHECK > 0	Size in bytes
MimeType	VARCHAR(50)	NOT NULL, DEFAULT 'application/pdf'	File type
UploadedBy	INT	NOT NULL, FK $\rightarrow$ Member	Uploader reference
FolderID	INT	NOT NULL, FK $\rightarrow$ Folder	Containing folder
IsConfidential	BOOLEAN	NOT NULL, DEFAULT FALSE	Confidentiality flag
IsActive	BOOLEAN	NOT NULL, DEFAULT TRUE	Soft-delete flag
CreatedAt	DATETIME	NOT NULL, DEFAULT NOW	Upload timestamp
UpdatedAt	DATETIME	NOT NULL, auto-update	Last modification time

2.2.6 DocumentVersion

Attribute	Type	Constraints	Description
VersionID	INT	PK, AUTO_INCREMENT	Unique version identifier
DocumentID	INT	NOT NULL, FK → Document	Parent document
VersionNumber	INT	NOT NULL, CHECK > 0	Sequence number
FilePath	VARCHAR(500)	NOT NULL	Versioned file path
FileSize	BIGINT	NOT NULL, CHECK > 0	Size in bytes
UploadedBy	INT	NOT NULL, FK → Member	Who uploaded this version
ChangeNote	VARCHAR(300)	–	Description of changes
CreatedAt	DATETIME	NOT NULL, DEFAULT NOW	Version timestamp

Unique Constraint: (DocumentID, VersionNumber) – ensures no duplicate version numbers per document.
Referential Integrity: DocumentID ON DELETE CASCADE (versions are removed with the document).

2.2.7 Tag

Attribute	Type	Constraints	Description
TagID	INT	PK, AUTO_INCREMENT	Unique tag identifier
TagName	VARCHAR(50)	NOT NULL, UNIQUE	Tag label
Description	VARCHAR(255)	–	Optional description
CreatedAt	DATETIME	NOT NULL, DEFAULT NOW	Creation timestamp

2.2.8 DocumentTag

Attribute	Type	Constraints	Description
DocumentID	INT	PK, FK → Document	Document reference
TagID	INT	PK, FK → Tag	Tag reference
AssignedAt	DATETIME	NOT NULL, DEFAULT NOW	When tag was applied

Composite Primary Key (DocumentID, TagID). Both FKs use ON DELETE CASCADE.

2.2.9 Permission

Attribute	Type	Constraints	Description
PermissionID	INT	PK, AUTO_INCREMENT	Unique permission ID
DocumentID	INT	NOT NULL, FK → Document	Target document
MemberID	INT	NOT NULL, FK → Member	Grantee
AccessLevel	ENUM	NOT NULL (VIEW, EDIT, DELETE, ADMIN)	Permission level
GrantedBy	INT	NOT NULL, FK → Member	Who granted it
GrantedAt	DATETIME	NOT NULL, DEFAULT NOW	Grant timestamp
ExpiresAt	DATETIME	CHECK: NULL or > GrantedAt	Optional expiry

Unique Constraint: (DocumentID, MemberID, AccessLevel) – prevents duplicate grants.
Logical Constraint: ExpiresAt must be NULL (permanent) or strictly after GrantedAt.

2.2.10 AccessLog

Attribute	Type	Constraints	Description
LogID	INT	PK, AUTO_INCREMENT	Unique log entry ID
DocumentID	INT	NOT NULL, FK → Document	Accessed document
MemberID	INT	NOT NULL, FK → Member	Who performed the action
Action	ENUM	NOT NULL	Action type (VIEW, DOWNLOAD, etc.)
IPAddress	VARCHAR(45)	–	Client IP address
UserAgent	VARCHAR(255)	–	Browser/client info
AccessTimestamp	DATETIME	NOT NULL, DEFAULT NOW	When it happened

Actions: VIEW, DOWNLOAD, UPLOAD, EDIT, DELETE, SHARE, PERMISSION_CHANGE.

2.2.11 SharedLink

Attribute	Type	Constraints	Descriptio
LinkID	INT	PK, AUTO_INCREMENT	Unique link identifier
DocumentID	INT	NOT NULL, FK → Document	Shared document
SharedBy	INT	NOT NULL, FK → Member	Link creator
Token	VARCHAR(128)	NOT NULL, UNIQUE	Unique access token
ExpiresAt	DATETIME	NOT NULL, CHECK > CreatedAt	Expiry timestamp
MaxDownloads	INT	NOT NULL, CHECK > 0, DEFAULT 5	Download limit
DownloadCount	INT	NOT NULL, CHECK ≥ 0 and ≤ MaxDownloads	Current count
IsActive	BOOLEAN	NOT NULL, DEFAULT TRUE	Active status
CreatedAt	DATETIME	NOT NULL, DEFAULT NOW	Creation timestamp

Logical Constraints:

- ExpiresAt > CreatedAt (link must expire in the future)
- 0 ≤ DownloadCount ≤ MaxDownloads (cannot exceed limit)

2.2.12 Notification

Attribute	Type	Constraints	Description
NotificationID	INT	PK, AUTO_INCREMENT	Unique notification ID
MemberID	INT	NOT NULL, FK → Member	Recipient
DocumentID	INT	FK → Document	Related document (nullable)
Message	VARCHAR(500)	NOT NULL	Notification text
NotificationType	ENUM	NOT NULL	Type category
IsRead	BOOLEAN	NOT NULL, DEFAULT FALSE	Read status
CreatedAt	DATETIME	NOT NULL, DEFAULT NOW	Notification timestamp

Types: SHARE, PERMISSION, UPDATE, UPLOAD, DELETE, SYSTEM.
Referential Integrity: DocumentID ON DELETE SET NULL (notification persists even if document is removed).

2.3 Summary of Constraints

Table 2: Constraint Compliance Checklist

Requirement	Status
Member table with MemberID (PK), Name, Image, Age, Email, ContactNumber	
At least 5 core functionalities	(6)
At least 5 distinct entities	(7)
At least 10 tables	(12)
Every table has a PK or uniquely identifiable candidate key	
Foreign Keys for all inter-table relationships	
Real-life synthetic data with 10–20 rows per table	
Referential integrity on UPDATE and DELETE	
At least 3 NOT NULL columns per table	
Unique row identification in every table	
Logical domain constraints (age $\geq$ 18, expiry $>$ grant, etc.)	

2.4 Key Logical Constraints

1. **Member.Age $\geq$ 18**: All system users must be adults.
2. **Document.FileSize** $>$ 0: Empty files are not permitted.
3. **DocumentVersion.VersionNumber** $>$ 0: Versions start at 1.

4. **Permission.ExpiresAt IS NULL OR ExpiresAt > GrantedAt**: Permissions cannot expire before they are granted.
5. **SharedLink.ExpiresAt > CreatedAt**: Links must have a future expiry.
6. **SharedLink.DownloadCount ≤ MaxDownloads**: Downloads cannot exceed the configured limit.
7. **SharedLink.MaxDownloads > 0**: A shared link must allow at least one download.

2.5 Normalisation

All tables satisfy **Third Normal Form (3NF)**:

- **1NF**: Every attribute holds atomic values; no repeating groups. Tags are normalised into a separate table with a junction (DocumentTag).
- **2NF**: No partial dependencies. The only composite key (DocumentTag) has no non-key attributes that depend on only part of the key; `AssignedAt` depends on the full composite key.
- **3NF**: No transitive dependencies. For instance, department details are stored in Department (not redundantly in Member), and role capabilities are in Role (not in Member).

3 Module B: Conceptual Design (UML and ER Diagrams)

3.1 UML Class Diagram

The following UML class diagram models the SafeDocs system. Each class represents an entity with its attributes and key methods. Relationships show multiplicity and type (association, composition, aggregation).

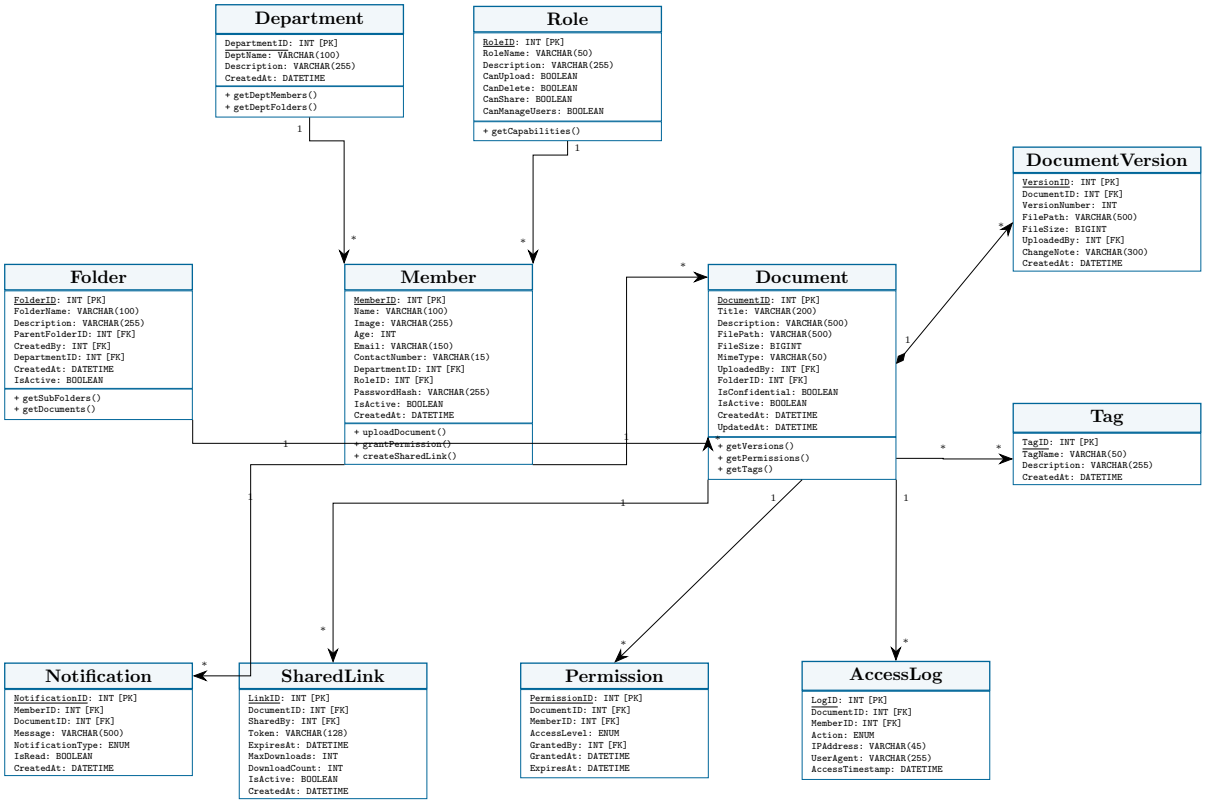

Figure 1: UML Class Diagram for SafeDocs

3.1.1 UML Relationship Summary

Table 3: UML Relationships and Multiplicity

From	To	Multiplicity	Type & Justification
Department	Member	1 : *	Association – a department has many members
Role	Member	1 : *	Association – a role is assigned to many members
Member	Document	1 : *	Association – a member uploads many documents
Folder	Document	1 : *	Aggregation – a folder contains many documents
Document	DocumentVersion	1 : *	Composition – versions cannot exist without the document
Document	Tag	* : *	Association – via DocumentTag junction
Document	Permission	1 : *	Association – a document has many permission entries
Member	Permission	1 : *	Association – a member can have many permissions
Document	AccessLog	1 : *	Association – a document has many log entries
Member	AccessLog	1 : *	Association – a member generates many log entries
Document	SharedLink	1 : *	Association – a document can have many shared links
Member	SharedLink	1 : *	Association – a member creates many shared links
Member	Notification	1 : *	Association – a member receives many notifications
Document	Notification	1 : *	Association – a document triggers many notifications
Folder	Folder	1 : *	Self-association – parent–child hierarchy

3.2 ER Diagram

The ER diagram below represents the conceptual schema derived from the UML class diagram. Entities are shown as rectangles, relationships as diamonds, and attributes as ellipses. Primary key attributes are <u>underlined</u>.

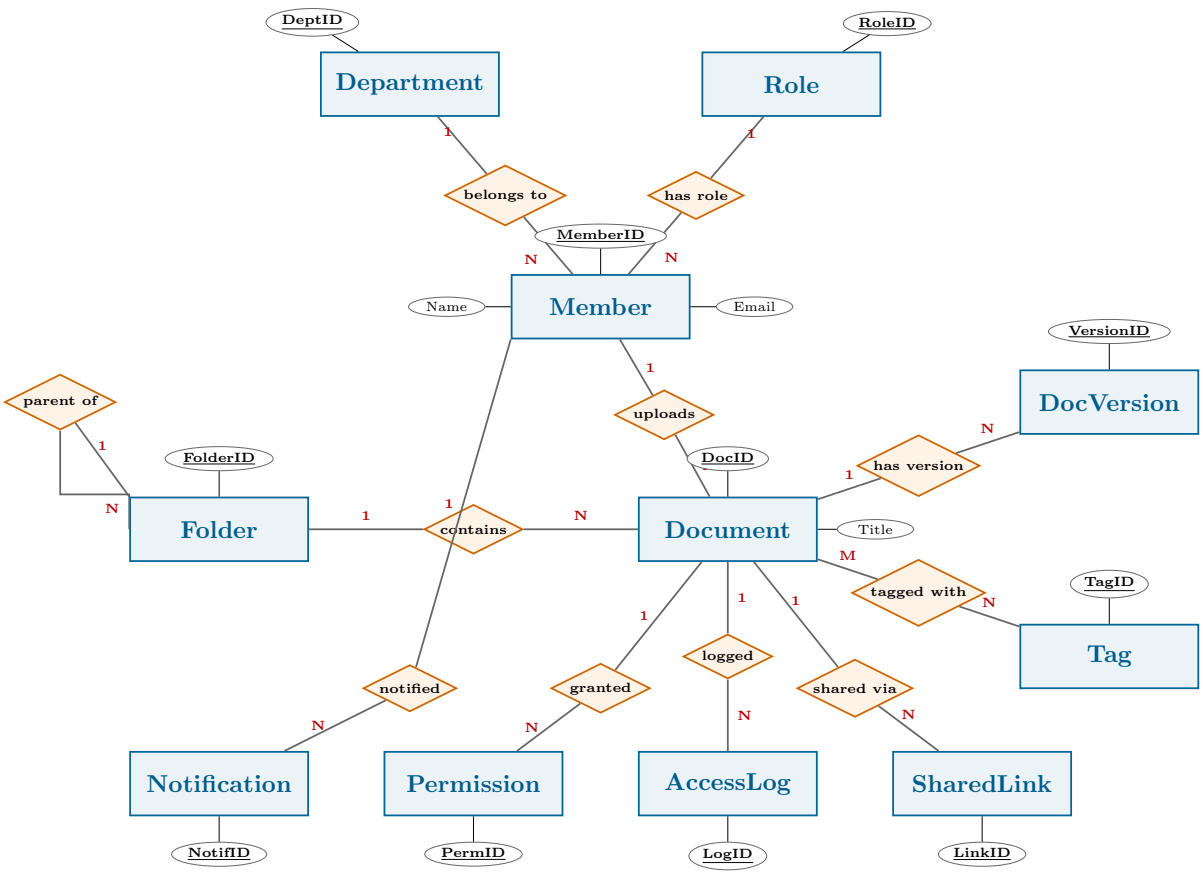

Figure 2: ER Diagram for SafeDocs

3.3 UML-to-ER Transition Explanation

The ER diagram was systematically derived from the UML class diagram as follows:

1. **Classes → Entities**: Each UML class (Department, Role, Member, Folder, Document, Tag, SharedLink, Notification) was mapped directly to a strong entity in the ER model. `DocumentVersion` became a *weak entity* identified by its parent Document.
2. **Attributes → ER Attributes**: All class attributes were preserved. Primary keys are underlined in the ER diagram. Methods were not carried over (they model application behaviour, not stored data).
3. **Associations → Relationships**: Each UML association was converted to an ER relationship diamond:
   - 1:* associations (Department–Member, Role–Member, Member–Document, Folder–Document) are realised via foreign keys on the "many" side.
   - The *:* association (Document–Tag) required the junction entity `DocumentTag`.
4. **Composition → Weak Entity**: The UML composition between Document and DocumentVersion indicates that versions cannot exist independently. In the ER model, DocumentVersion is a weak entity with a partial key (VersionNumber) and total participation via the "has version" relationship. This maps to ON DELETE CASCADE in SQL.

5. **Self-Association → Recursive Relationship**: The Folder self-association (parent–child) becomes a recursive "parent of" relationship in ER, realised by the ParentFolderID foreign key referencing the same table.
6. **Multiplicity → Cardinality**: UML multiplicities were directly translated:
   - `1` in UML → `1` in ER (e.g., one department)
   - `*` in UML → `N` in ER (e.g., many members)
   - `* : *` in UML → `M : N` in ER (e.g., Document–Tag)

3.4 Relationship Justifications

1. **Department** $\xrightarrow{1:N}$ **Member**: A department has many members, but each member belongs to exactly one department. *Example*: "Computer Science" department has Aarav, Priya, and Karan as members.
2. **Role** $\xrightarrow{1:N}$ **Member**: Each member has exactly one role, but a role can be assigned to many members. *Example*: The "Editor" role is held by Ananya, Sneha, Arjun, and others.
3. **Member** $\xrightarrow{1:N}$ **Document**: A member can upload many documents, but each document has exactly one uploader. *Example*: Priya uploaded "ML Research Paper – CNN" and "Research Grant Proposal".
4. **Folder** $\xrightarrow{1:N}$ **Document**: A folder contains many documents; each document belongs to one folder. *Example*: "Exam Papers" folder contains "CS432 Database Syllabus" and "CS432 Midterm Paper 2024".
5. **Folder** $\xrightarrow{1:N}$ **Folder (self)**: A folder can have many sub-folders, but each sub-folder has at most one parent. *Example*: "CS Department Docs" is the parent of "Exam Papers", "Research Papers", and "Faculty Meeting Minutes".
6. **Document** $\xrightarrow{1:N}$ **DocumentVersion (composition)**: A document has many versions; versions are existentially dependent on the document. *Example*: "Research Grant Proposal" has v1 (first draft), v2 (revised budget), v3 (final with co-PI).
7. **Document** $M : N$ **Tag**: A document can have many tags and a tag can label many documents. Realised via `DocumentTag`. *Example*: "Annual Budget Report FY25" is tagged both "Finance" and "Confidential".
8. **Document** $\xrightarrow{1:N}$ **Permission**: A document can have multiple permission entries for different members. *Example*: "CS432 Database Syllabus" has ADMIN for Aarav, EDIT for Priya, and VIEW for Kavya.
9. **Document** $\xrightarrow{1:N}$ **SharedLink**: A document can have multiple shared links (e.g., different links for different audiences with different limits). *Example*: "Student Handbook" has a link with 50 max downloads.
10. **Member** $\xrightarrow{1:N}$ **Notification**: A member receives many notifications. *Example*: Priya received update notifications for both v2 and v3 of the "Research Grant Proposal".

4 Additional Constraints and Design Decisions

1. **Soft Deletes**: Documents and folders use an `IsActive` flag instead of physical deletion, preserving referential integrity and audit trails.

2. **Immutable Audit Logs**: The `AccessLog` table is append-only by design. No UPDATE or DELETE operations should be permitted on this table in production.
3. **Token Uniqueness**: `SharedLink.Token` has a UNIQUE constraint to prevent token collisions.
4. **Cascading Deletes**: Version history and tags cascade on document deletion (data that has no meaning without the parent document). Permissions and access logs also cascade.
5. **Restrict Deletes**: Member deletion is restricted if the member has uploaded documents or created folders, preventing orphaned records.
6. **Notification Persistence**: `Notification.DocumentID` is SET NULL on document deletion so that the notification message is preserved for audit purposes even if the referenced document is removed.
7. **Index Strategy**: Indexes are placed on all foreign key columns and on `AccessLog.AccessTimesta` for efficient temporal queries.
8. **Password Storage**: The `PasswordHash` field stores bcrypt-hashed passwords (never plaintext) for security compliance.

5 Sample Data Overview

The SQL dump includes realistic synthetic data:

Table 4: Sample Data Row Counts

Table	Rows
Department	15
Role	5
Member	20
Folder	15
Document	20
DocumentVersion	15
Tag	12
DocumentTag	20
Permission	20
AccessLog	20
SharedLink	12
Notification	20

All data reflects an institutional context (IIT campus) with meaningful department names, realistic document titles, logical folder hierarchies, and consistent foreign key references.

6 Team Member Contributions

Name	Roll No.	Contribution
Sujal Patel	22110261	Schema design, SQL implementation, report compilation
Siddharth Doshi	22110250	ER diagram design, relationship analysis, data validation
Viraj Vekaria	22110287	UML class diagram, constraint specification, testing
Nishit Mistry	22110172	Sample data generation, normalisation verification
Rutvi Shah	22110227	UML-to-ER mapping, documentation, report formatting

7 References

1. R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th Edition. Chapter 7 (ER Model), Chapter 10 (UML).
2. A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th Edition. Chapter 6 (ER Model).
3. MySQL 8.0 Reference Manual – https://dev.mysql.com/doc/refman/8.0/en/